

Homework 2 - Bias-Variance Tradeoff & Linear Methods

Instructions

Due October 9 at 11pm

Instructions

- Use the space inside of

```
::: {.solbox data-latex=""}
```

```
:::
```

to answer the following questions.

- Do not move this file or copy it and work elsewhere. Work in the same directory.
- Use a new branch named whatever you want. Create it now! Make a small change, say by adding your name. Commit and push now!
- Try to Knit your file now. Are there errors? Fix them now, not at 11pm on the due date.
- There MUST be some text between `::: {.solbox}` and the next `:::` or this will fail to Knit.
- If your code or figures run off the edge of the `.pdf`, you'll lose 2 points automatically.
- Be sure to add your name in the `author` field at the top.

Generative AI Self Report

If you use any generative AI tool on homeworks or labs, indicate here how you used the tools and how they contributed to your learning/ability to understand the assignment. Include every prompt that you used.

This self report will not affect your grade in any way. Please be as honest as possible.

- your response here *

1 Regularized Methods & Linear Models (4 points)

In this section we will conduct an analysis of `mobility`, a dataset measuring economic mobility across generations in the contemporary USA. The data come from a large study, based on tax records, which allowed researchers to link the income of adults to the income of their parents several decades previously. For privacy reasons, we don't have that individual-level data, but we do have aggregate statistics about economic mobility for several hundred communities, containing most of the American population, and covariate information about those communities. We are interested in predicting economic mobility from the characteristics of communities. Note that some observations are missing values for some covariates.

Use `?mobility` in the console or visit the package [website](#) for information on the variables contained here.

1.1 (1 point)

Make scatter plots of mobility against each of the following variables (by “plot mobility against B” we mean “put mobility on the y-axis and B on the x-axis”): `Population`, `Income`, `Seg_racial`, `Share01`, `School_spending`, `Violent_crime`, and `Commute`. Include on each plot a line for the univariate regression of mobility on the variable, and give a table of the slope coefficients. Carefully explain the interpretation of each coefficient in the context of the problem. You likely need to look at the documentation to determine what these variables mean. **Hint:** This can be done easily with `ggplot() + geom_smooth()`. See the Examples [here](#). If you choose a different route, I suggest writing a function.

(placeholder text)

1.2 (1 point)

In this question, we will run a linear regression of `mobility` against all appropriate covariates and answer some questions. By “appropriate”, we mean

use any covariate that could potentially be predictive of mobility. Some are obviously useless or colinear with others. Don’t use those. All that is necessary to make this choice is to examine the documentation for the data `?mobility` or [on the web](#).

- (0.25 points) Report all regression coefficients and their standard errors to reasonable precision; you may use either a table or a figure as you prefer. Do not just paste in R’s output.
- (0.25 points) Explain why the `ID` variable must be excluded.
- (0.25 points) Explain which other variables, if any, you excluded from the regression, and why. (If you think they can all be used, explain why.)
- (0.25 points) Compare the coefficients you found in Question 1.1 to the coefficients for the same variables in this regression. Are they much different? Have any changed sign?

```
ols <- lm(Mobility ~ . , data = mobility) # YOU NEED TO CHANGE THIS LINE!!
```

```
# some code
```

- a.
- b.
- c.
- d.

1.3 (0.5 point)

Continuing with all the covariates you used in the previous question, use ridge regression and lasso (with the `{glmnet}` package). Use cross validation as implemented in `cv.glmnet()`.

- (0.25 points) Plot the CV curve for Lasso. Explain the difference between the two vertical lines shown on the figure. What are the numbers on the top of the plot?
- (0.25 points) Plot the coefficient trace for ridge regression. As $\lambda \rightarrow 0$, which way do I move on the x-axis?

a.

```
library(glmnet)
x <- model.matrix(ols) # grabs the design matrix from the internal lm() output
x <- x[, -1] # remove the intercept column, glmnet() will do this for us
y <- mobility$Mobility[-ols$na.action] # remove those observations with
# missing values. glmnet() hates those. They are already dropped in x.
set.seed(01101101)
```

b.

```
# more code
```

1.4 (0.5 points)

For both the Ridge regression and Lasso regression in Question 1.3, choose the set of coefficient estimates using `lambda.min`. Compare these coefficient estimates with those in Question 1.2 by producing a graph (put the Variable names on the y-axis, the coefficient estimate on the x-axis, and use colour or shape to denote the three methods). Describe what you see.

```
# some code
```

(placeholder text)

1.5 (1 point)

We can get the CV scores for the `lambda.min` lasso and ridge models from Question 1.3 via:

```
# set eval=TRUE on this block to run this code
tibble(
  method = c("lasso", "ridge"),
  cv.score = c(
    lasso$cvm[lasso$lambda == lasso$lambda.min],
    ridge$cvm[ridge$lambda == ridge$lambda.min]
  )
)
```

- (0.5 points) Based on these scores, will (tuned) lasso or ridge get lower prediction error in expectation on this particular train/test split? Justify your answer.
- (0.5 points) Based on these scores, will (tuned) lasso or ridge get lower prediction error in expectation on a different train/test split? Justify your answer.

- a.
- b.

2 The Bias-Variance Tradeoff (6 points)

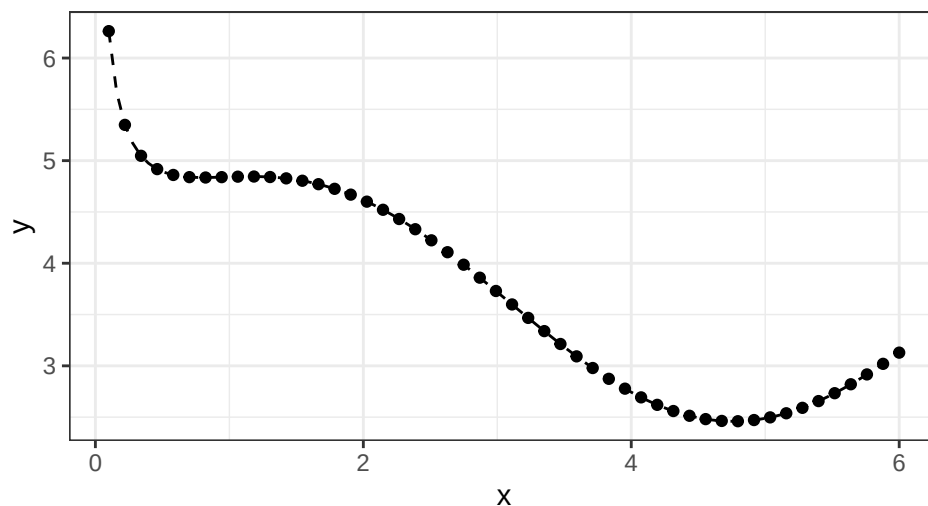
In this question, you will explore the bias-variance tradeoff of polynomial regression on the `wiggly_function`. (Run `?wiggly_function` to see the function.)

To approximate risk and the bias/variance components. - Generate a fixed test set, which we will use to evaluate trained polynomial regressors. - Generate ~100 noisy training samples, which we will train polynomial regressors on. - We will estimate risk, bias, and variance using the training samples and the test set to approximate the expectations $E[(Y - E[\hat{Y}(X) | X])^2]$ and $E[(\hat{Y}(X) - E[\hat{Y}(X) | X])^2]$.

First, run the following code that creates `test.set` and plots the true function (as a line) and the training data (as points).

```
test.set <- tibble(x = seq(0.1, 6, length.out = 50))
test.set$y <- wiggly_function(test.set$x)
test.set %>%
  ggplot(aes(x, y)) +
  geom_point() +
  stat_function(fun = wiggly_function, linewidth = 0.5, linetype = "dashed") +
  labs(title = "Wiggly Test Set (+ Wiggly Function)", x = "x", y = "y") +
  theme_bw()
```

Wiggly Test Set (+ Wiggly Function)



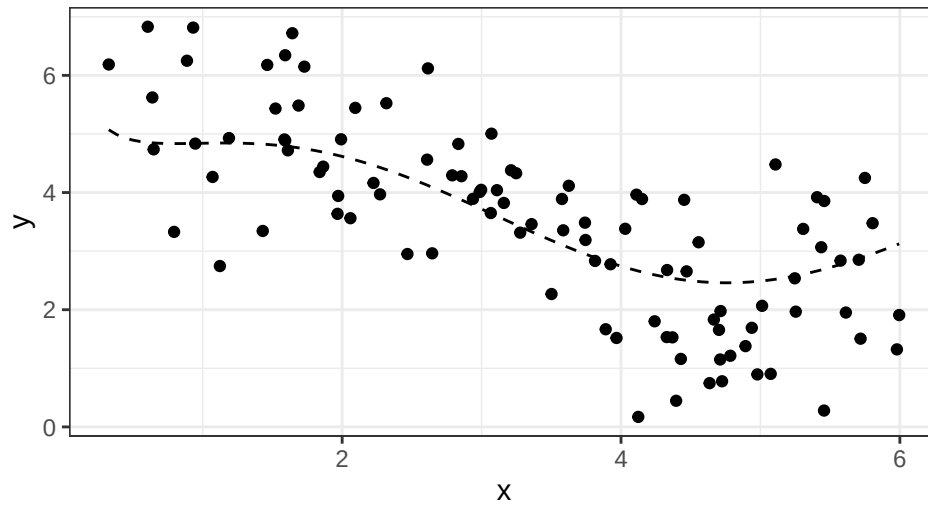
You will also use this function to generate training datasets.

```
create.training.sample <- function(n = 100) {
  # Generate random x values between 0.1 and 6
  x <- runif(n, min = 0.1, max = 6)
  # The response will be the wiggly function with added Gaussian noise
  y <- wiggly_function(x) + rnorm(n, sd = 1.0)
  # Return a data frame with x and y
  tibble(x = x, y = y)
}

# Plot a sample training set
create.training.sample() %>%
  ggplot(aes(x, y)) +
  geom_point() +
  stat_function(fun = wiggly_function, linewidth = 0.5, linetype = "dashed") +
  labs(title = "Training Sample (+ Wiggly Function)", x = "x", y = "y") +
```

```
theme_bw()
```

Training Sample (+ Wiggly Function)



2.1 (1 point)

- (0.5 points) What differences do you see between the test set and the training sample?
- (0.5 points) Given these differences, is it reasonable to expect that a regression model fit on the training data will generalize to the test data? Explain in 2-3 sentences.

- a.
- b.

2.2 (1 point)

We will first create a simple function for training and evaluating a polynomial regression model on arbitrary train/test data. (In the next sub-problem, we will apply this function to multiple training samples to simulate the bias-variance tradeoff.)

- a. (0.5 points) Complete the function `compute.polynomial.predictions` below that trains a polynomial regression model on a training dataset and outputs predictions on some test dataset. (Hint: use the `?poly` function to create polynomial features.)
- b. (0.5 points) Test this function with:
 - `train.data` is a training sample (from `create.training.data`) of size 25
 - `degree` is 3
 - `test.data` is `test.set`

Plot the resulting predictions against the true `test.set` values.

```
a.  
  
# ' Trains a polynomial regression model on training data and outputs  
# ' predictions on test data.  
# ' @param train.data A data frame containing a training sample with  
# ' columns `x` and `y`.  
# ' @param test.data A data frame containing the test data (e.g. `test.set`)  
# ' with columns `x`.  
# ' @param degree An integer specifying the degree of the polynomial to fit.  
# ' @return A vector containing the predictions for the test data.  
compute.polynomial.predictions <- function(train.data, test.data, degree) {  
  # TODO: Fit a polynomial model of the specified degree on train.data  
  # TODO: Return the predictions of the polynomial regression model  
}
```

b.

2.3 (2 points)

We will now use the function above to collect the data we need to simulate the bias-variance tradeoff. In particular, we will get predictions from polynomial models of varying complexity (degrees 1-10) trained on numerous (simulated) training samples of size 100. We will use these results in the next section to estimate the bias and variance of polynomial regression as a function of the polynomial degree.

- a. (1.5 points) Run `compute.polynomial.predictions` on 100 different $n = 100$ training samples from `create.training.sample`, using polynomial degrees from 1 to 10 (inclusive). For each combination of split and degree, store the predictions on the `test.set` data. Store your results in a dataframe called `wiggly.preds` with columns: `split.number`, `degree`, `x`, `y.hat`. The final dataframe should have $100 \text{ splits} \times 10 \text{ degrees} \times 50 \text{ test points} = 50,000$ rows.

Important: before making any calls to `create.training.sample`, ensure the seed is locked to 12345 so that your results are reproducible.

- b. (0.5 points) Computing both bias and variance requires “average model predictions,” where we average out the randomness in the training sample. Construct a dataframe called `wiggly.avg.preds` that averages the test set predictions across all training sample simulations. (Note that you **should not** average over test data points or polynomial degree. You should end up with a dataframe with 500 rows.)

a.

```
set.seed(12345) # Lock the seed so results are reproducible  
  
# your code
```

b.

2.4 (2.5 points)

We now have the simulation data we need to estimate the bias and variance of polynomial regression models on the `wiggly` dataset. We will see how these two terms change as we increase the polynomial degree.

- a. (0.5 points) We are only going to decompose the risk into two terms (squared bias and variance), and we will not include the third term (irreducible noise) that we discussed in lecture. Why? Explain in 1-2 sentences.
- b. (1 point) Construct a dataframe called `wiggly.bias.var` that contains the squared bias, variance, and total risk of a polynomial regressor as a function of the polynomial degree.
- c. (0.5 points) Plot the bias, variance, and total risk as a function of polynomial degree. In 2 sentences, describe the trends you see.
- d. (0.5 points) Play around with the different variables in this simulation. What happens to bias and variance if you use smaller samples of your training data? Why? Answer in ~2 sentences.

- a.
- b.
- c.
- d.